chiselWare

MAC
Product User Guide

https://github.com/shreosidebnath/ClockDomainCrew-MAC

March 23, 2026

Contents

1 Errata and Known Issues **3**
1.1 Errata . . . 3
1.2 Known Issues . . . 3

2 Port Descriptions **4**
2.1 AXI-Stream Interface Signals . . . 8
2.2 Interface Configuration Summary . . . 8

3 Parameter Descriptions **9**
3.1 Example Instantiation . . . 9

4 Theory of Operation **10**
4.1 Overview . . . 10
4.2 Architecture . . . 10
4.3 Transmit Path Logic . . . 11
4.3.1 Ethernet Frame Format . . . 11
4.3.2 Transmit State Machine . . . 11
4.3.3 Frame Padding . . . 12
4.3.4 Parallel CRC Generation . . . 12
4.3.5 Inter-Frame Gap (IFG) . . . 12
4.3.6 Deficit Idle Count (DIC) . . . 12
4.3.7 Lane Alignment and Swapping . . . 12
4.4 Receive Path Logic . . . 12
4.4.1 Receive State Machine . . . 13
4.4.2 Frame Detection and Preamble Validation . . . 13
4.4.3 Cross-Cycle Realignment . . . 14
4.4.4 CRC Residue Verification . . . 14
4.4.5 Packet Classification . . . 14
4.5 Packet Statistics . . . 14
4.6 Precision Time Protocol (PTP) . . . 15
4.7 Integration . . . 15

5 Simulation **17**
5.1 Tests . . . 17
5.2 Code coverage . . . 17
5.3 Running simulation . . . 17

6 Synthesis **18**
6.1 Area . . . 18
6.2 SDC File . . . 18
6.3 Timing . . . 18
6.4 Multicycle Paths . . . 18

1 Errata and Known Issues

This section tracks the current limitations and known deviations from the full Ethernet specification for the **Mac** core.

1.1 Errata

There are currently no known functional bugs (errata) that deviate from the intended design of the 10GbE 64-bit architecture.

1.2 Known Issues

The following items are known limitations in the current release of the **Mac** core:

- **Strict 64-bit Architecture:** The **Mac** core is strictly architected for 10GbE 64-bit data paths. While the *dataW* parameter is present in the configuration class, the internal submodules *Xgmii2Axis64* and *Axis2Xgmii64* are strictly architected for a 64-bit data path. The *dataW* parameter is guarded by a Scala `require` statement. Attempting to elaborate the design with $dataW \neq 64$ will result in a elaboration-time error. The parameter is retained in the configuration for potential future support of different Ethernet data widths, but currently only 64-bit operation is supported.
- **Control Frame Logic Bypassed:** The optional MAC Control Frame (MCF) insertion and extraction logic is currently bypassed.
  - The **sAxisTx** and **mAxisRx** user interfaces are connected directly to the internal *Axis2Xgmii64* and *Xgmii2Axis64* submodules via an *AxisTie* bridge.
  - As a result, Link Flow Control (LFC) and Priority Flow Control (PFC) requests are not processed, and pause frames are neither generated nor interpreted.
  - All control-related status outputs are tied to logic **'0'** to ensure stability in the statistics aggregator.
- **Inactive Configuration Ports:** Configuration inputs related to Flow Control (prefixed with *cfgMcf*, *cfgTxLfc*, and *cfgTxPfc*) are currently disconnected from the data path and have no effect on frame processing.

2 Port Descriptions

The ports for **Mac** are shown below in Table 1. The width of several ports is controlled by the following parameters:

- *dataW* is the width of the XGMII data bus in bits
- *ctrlW* is the width of the XGMII control bus in bits, which must be equal to *dataW*/8 for byte granularity
- *gbxCnt* is the width of the internal counter used for tracking the number of bytes in a frame, which must be wide enough to count up to the maximum frame size (e.g., 12 bits for 4096 bytes)
- *ptpTsW* is the width of the PTP timestamp bus in bits, which must be sufficient to represent the desired timestamp resolution and range (e.g., 48 bits for nanosecond resolution over several years)

Port Name	**Width**	**Direction**	**Description**
Clock and Reset Signals			
rxClk	1	Input	RX domain clock
rxRst	1	Input	RX domain active-high synchronous reset
txClk	1	Input	TX domain clock
txRst	1	Input	TX domain active-high synchronous reset
statClk	1	Input	Statistics domain clock
statRst	1	Input	Statistics domain active-high synchronous reset
User Side AXI-Stream Interfaces			
sAxisTx	See Table 2	**Sink** (Slave)	**TX Sink:** User data to be transmitted
mAxisTxCpl	See Table 2	**Source** (Master)	**TX Completion:** Status and PTP timestamps
mAxisRx	See Table 2	**Source** (Master)	**RX Source:** Received Ethernet frames to user
mAxisStat	See Table 2	**Source** (Master)	**Stats Source:** Aggregated statistics update bus
Physical Layer Interface (XGMII)			
xgmiiRxd	*dataW*	Input	XGMII receive data bus
xgmiiRxc	*ctrlW*	Input	XGMII receive control bus
xgmiiRxValid	1	Input	XGMII receive data valid
xgmiiTxd	*dataW*	Output	XGMII transmit data bus
xgmiiTxc	*ctrlW*	Output	XGMII transmit control bus
xgmiiTxValid	1	Output	XGMII transmit data valid
Gearbox and Synchronization			
txGbxReqSync	*gbxCnt*	Input	Gearbox synchronization request from PCS
txGbxReqStall	1	Input	Gearbox stall request (pauses MAC)
txGbxSync	*gbxCnt*	Output	Gearbox synchronization acknowledgment

Port Name	Width	Direction	Description
Precision Time Protocol (PTP)			
txPtpTs	*ptpTsW*	Input	Reference timestamp for TX SOF capture
rxPtpTs	*ptpTsW*	Input	Reference timestamp for RX SOF capture
Link-Level Flow Control (LFC)			
txLfcReq	1	Input	Trigger LFC frame transmission
txLfcResend	1	Input	Retransmit current LFC pause frame
rxLfcEn	1	Input	Enable LFC frame reception/processing
rxLfcReq	1	Output	Asserted when an LFC pause is received
rxLfcAck	1	Input	Acknowledge LFC pause request
Priority Flow Control (PFC)			
txPfcReq	8	Input	Trigger PFC for 8 priority classes
txPfcResend	1	Input	Retransmit current PFC pause frame
rxPfcEn	8	Input	Enable PFC for 8 priority classes
rxPfcReq	8	Output	Asserted when a PFC pause is received
rxPfcAck	8	Input	Acknowledge PFC pause request
Pause Interface			
txLfcPauseEn	1	Input	Enable automatic TX pause on LFC
txPauseReq	1	Input	Trigger standard 802.3x pause frame
txPauseAck	1	Output	Acknowledge pause frame transmission
Transmit Status and Monitoring			
txStartPacket	2	Output	Pulse indicating SOF (lane aligned)
statTxByte	4	Output	Valid bytes in current TX cycle
statTxPktLen	16	Output	Length of last transmitted packet
statTxPktUcast	1	Output	Last TX packet was Unicast
statTxPktMcast	1	Output	Last TX packet was Multicast
statTxPktBcast	1	Output	Last TX packet was Broadcast
statTxPktVlan	1	Output	Last TX packet was VLAN tagged
statTxPktGood	1	Output	Last TX packet was valid
statTxPktBad	1	Output	Last TX packet had errors
statTxErrOversize	1	Output	TX Packet exceeded MTU
statTxErrUser	1	Output	TX Error triggered by user (tuser)
statTxErrUnderflow	1	Output	TX Error: sAxisTx underflowed
Receive Status and Monitoring			
rxStartPacket	2	Output	Pulse indicating SOF (lane aligned)
statRxByte	4	Output	Valid bytes in current RX cycle

Port Name	Width	Direction	Description
statRxPktLen	16	Output	Length of last received packet
statRxPktFragment	1	Output	Received packet was a fragment (<64B)
statRxPktJabber	1	Output	Received packet was a jabber (>MTU)
statRxPktUcast	1	Output	Last RX packet was Unicast
statRxPktMcast	1	Output	Last RX packet was Multicast
statRxPktBcast	1	Output	Last RX packet was Broadcast
statRxPktVlan	1	Output	Last RX packet was VLAN tagged
statRxPktGood	1	Output	Last RX packet passed FCS check
statRxPktBad	1	Output	Last RX packet failed FCS check
statRxErrOversize	1	Output	RX Packet exceeded MTU
statRxErrBadFcs	1	Output	RX Packet had a bad FCS residue
statRxErrBadBlock	1	Output	RX Packet contained 64b/66b error blocks
statRxErrFraming	1	Output	RX Packet had XGMII framing errors
statRxErrPreamble	1	Output	RX Packet had an invalid preamble
statRxFifoDrop	1	Input	External RX FIFO overflow indicator
Control Frame Statistics			
statTxMcf	1	Output	Transmitted MAC Control Frame
statRxMcf	1	Output	Received MAC Control Frame
statTxLfcPkt	1	Output	Transmitted LFC packet
statTxLfcXon	1	Output	Transmitted LFC XON (Pause 0)
statTxLfcXoff	1	Output	Transmitted LFC XOFF
statTxLfcPaused	1	Output	TX path is currently paused by LFC
statTxPfcPkt	1	Output	Transmitted PFC packet
statTxPfcXon	8	Output	Transmitted PFC XON per class
statTxPfcXoff	8	Output	Transmitted PFC XOFF per class
statTxPfcPaused	8	Output	TX path currently paused per PFC class
statRxLfcPkt	1	Output	Received LFC packet
statRxLfcXon	1	Output	Received LFC XON
statRxLfcXoff	1	Output	Received LFC XOFF
statRxLfcPaused	1	Output	RX path reports LFC pause state
statRxPfcPkt	1	Output	Received PFC packet
statRxPfcXon	8	Output	Received PFC XON per class
statRxPfcXoff	8	Output	Received PFC XOFF per class
statRxPfcPaused	8	Output	RX path reports PFC pause per class
Runtime Configuration			
cfgTxMaxPktLen	16	Input	Maximum frame length for TX

Port Name	Width	Direction	Description
cfgTxIfg	8	Input	Inter-Frame Gap (IFG) setting in bytes
cfgTxEnable	1	Input	Global TX path enable
cfgRxMaxPktLen	16	Input	Maximum frame length for RX
cfgRxEnable	1	Input	Global RX path enable
cfgMcfRxEthDstMcast	48	Input	MCF Multicast Destination MAC
cfgMcfRxCheckEthDstMcast	1	Input	Enable MCF Multicast address check
cfgMcfRxEthDstUcast	48	Input	MCF Unicast Destination MAC
cfgMcfRxCheckEthDstUcast	1	Input	Enable MCF Unicast address check
cfgMcfRxEthSrc	48	Input	MCF Source MAC address
cfgMcfRxCheckEthSrc	1	Input	Enable MCF Source address check
cfgMcfRxEthType	16	Input	MCF Ethernet Type (usually 0x8808)
cfgMcfRxOpcodeLfc	16	Input	Opcode for LFC frames
cfgMcfRxCheckOpcodeLfc	1	Input	Enable LFC Opcode checking
cfgMcfRxOpcodePfc	16	Input	Opcode for PFC frames
cfgMcfRxCheckOpcodePfc	1	Input	Enable PFC Opcode checking
cfgMcfRxForward	1	Input	Forward MCF frames to user logic
cfgMcfRxEnable	1	Input	Global MCF processing enable
cfgTxLfcEthDst	48	Input	TX LFC Destination MAC
cfgTxLfcEthSrc	48	Input	TX LFC Source MAC
cfgTxLfcEthType	16	Input	TX LFC Ethernet Type
cfgTxLfcOpcode	16	Input	TX LFC Opcode
cfgTxLfcEn	1	Input	Enable TX LFC generation
cfgTxLfcQuanta	16	Input	Pause duration in quanta for LFC
cfgTxLfcRefresh	16	Input	Refresh timer value for LFC
cfgTxPfcEthDst	48	Input	TX PFC Destination MAC
cfgTxPfcEthSrc	48	Input	TX PFC Source MAC
cfgTxPfcEthType	16	Input	TX PFC Ethernet Type
cfgTxPfcOpcode	16	Input	TX PFC Opcode
cfgTxPfcEn	1	Input	Enable TX PFC generation
cfgTxPfcQuanta	8×16	Input	Pause duration quanta per priority class
cfgTxPfcRefresh	8×16	Input	Refresh timer per priority class
cfgRxLfcOpcode	16	Input	Expected Opcode for incoming LFC
cfgRxLfcEn	1	Input	Enable incoming LFC processing
cfgRxPfcOpcode	16	Input	Expected Opcode for incoming PFC
cfgRxPfcEn	1	Input	Enable incoming PFC processing

Table 1: MAC Port Descriptions

2.1 AXI-Stream Interface Signals

All AXI-Stream interfaces in the **Mac** core follow the signal definitions shown in Table 2. Note that the width of some signals are determined by the specific *AxisInterfaceParams* configuration for that port.

Signal	**Width**	**Direction**[1]	**Description**
tdata	*dataW*	Output	Primary data bus
tkeep*	*keepW*	Output	Byte qualifiers; indicates valid bytes in *tdata*
tstrb*	*keepW*	Output	Position byte qualifiers (usually matches *tkeep*)
tid*	*idW*	Output	Stream identifier (used for TX tags/completion)
tdest*	*destW*	Output	Routing destination identifier
tuser*	*userW*	Output	Sideband metadata (used for PTP timestamps in RX)
tlast	1	Output	Indicates the last transfer of a frame
tvalid	1	Output	Indicates that the source is driving valid signals
tready	1	Input	Indicates that the sink can accept data

Table 2: AXI-Stream Signal Definitions

[1] Directions are shown for a **Source** (Master) interface. For **Sink** (Slave) interfaces, such as *sAxisTx*, all directions are reversed (Flipped).

2.2 Interface Configuration Summary

Table 3 summarizes the parameter-driven widths for each AXI-Stream interface instance.

Interface	**dataW**	**keepW**	**idW**	**userW**	**destW**
sAxisTx	*p.dataW*	*p.dataW / 8*	8	1	8
mAxisTxCpl	96	1	8	1	8
mAxisRx	*p.dataW*	*p.dataW / 8*	8	*rxUserW*[2]	8
mAxisStat	16	1	8	1	8

Table 3: Interface Parameter Mapping

[2] *rxUserW* is calculated as *p.ptpTsW + 1* if PTP is enabled.

3 Parameter Descriptions

The hardware parameters for the **Mac** core are described in Table 4. These parameters control the architectural features, data-path widths, and monitoring capabilities of the controller.

Name	**Type**	**Min**	**Max**	**Description**
dataW	Int	64	64	XGMII data bus width (Fixed for 10G)
ctrlW	Int	8	8	XGMII control bus width
txGbxIfEn	Boolean	false	true	Enable Gearbox interface for TX path
rxGbxIfEn	Boolean	false	true	Enable Gearbox interface for RX path
gbxCnt	Int	1	–	Width of gearbox synchronization bus
paddingEn	Boolean	false	true	Enable auto-padding of short frames
dicEn	Boolean	false	true	Enable Deficit Idle Count (DIC) for IFG
minFrameLen	Int	64	–	Minimum Ethernet frame size in bytes
ptpTsEn	Boolean	false	true	Enable Precision Time Protocol (PTP)
ptpTsFmtTod	Boolean	false	true	Timestamp Format (True: ToD, False: Binary)
ptpTsW	Int	1	128	Bit-width of the PTP timestamp bus
statEn	Boolean	false	true	Enable internal Statistics Aggregator
statTxLevel	Int	0	2	Transmit statistics granularity level
statRxLevel	Int	0	2	Receive statistics granularity level
statIdBase	Int	0	–	Base index for statistics reporting ID
statUpdatePeriod	Int	1	–	Period (cycles) between stats updates
statStrEn	Boolean	false	true	Enable human-readable metadata strings
statPrefixStr	String	–	–	Prefix for identifying the MAC instance

Table 4: Mac Parameter Descriptions

3.1 Example Instantiation

The **Mac** core is typically instantiated in Chisel as a ‘RawModule‘ to allow for multiple asynchronous clock domains. An example configuration with PTP enabled is shown below:

```
// Define a custom configuration
val myMacParams = MacParams(
  ptpTsEn       = true,
  ptpTsW        = 96,
  statPrefixStr = "WAN_PORT_0"
)

// Instantiate the MAC core
val ethernetMac = Module(new Mac(myMacParams))

// Connect clocks
ethernetMac.io.txClk    := coreClock
ethernetMac.io.rxClk    := phyClock
ethernetMac.io.statClk := systemClock
```

4 Theory of Operation

4.1 Overview

The **Mac** core is a high-performance 10GbE Media Access Controller implementation. It provides a full-duplex data path between a standard AXI-Stream user interface and the 64-bit XGMII physical layer interface for transmitting and receiving ethernet frames. The core is designed as a `RawModule` to support three independent asynchronous clock domains: *txClk*, *rxClk*, and *statClk*.

The MAC performs the following functions:

- Ethernet frame generation
- Ethernet frame reception
- Frame Check Sequence (FCS) generation and verification
- Frame padding and minimum frame length enforcement
- Inter-frame gap enforcement
- Packet statistics reporting
- Optional Precision Time Protocol (PTP) timestamp support

The design contains two primary datapaths:

- **Transmit Path (TX):** Converts AXI-Stream packets into Ethernet frames transmitted on XGMII.
- **Receive Path (RX):** Converts Ethernet frames received from XGMII into AXI-Stream packets.

4.2 Architecture

The top-level core integrates three primary functional blocks:

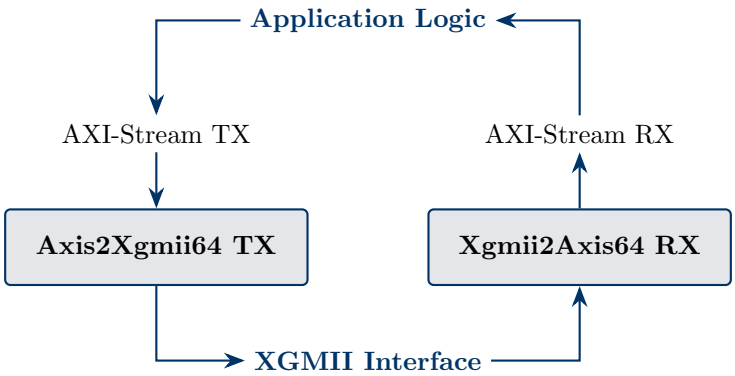

Figure 1: MAC Top-Level Architecture

- **Transmit Path (Axis2Xgmii64):** Encapsulates AXI-Stream packets into Ethernet frames with preamble, SFD, padding, and CRC-32.
- **Receive Path (Xgmii2Axis64):** Decapsulates XGMII data, validates frame integrity via CRC residue checking, and extracts timestamps.
- **Statistics Aggregator (MacStats):** A dedicated submodule that tracks over 30 distinct Ethernet metrics across the clock domains.

The user-side AXI-Stream interfaces are connected via *AxisTie* modules, ensuring robust signal bridging and default tie-offs for optional sideband signals like *tid* and *tdest*.

4.3 Transmit Path Logic

The transmit datapath converts AXI-Stream packets into Ethernet frames suitable for transmission over XGMII. Main responsibilities include:

- Frame construction
- Frame padding
- CRC generation
- Inter-frame gap enforcement

Transmit Pipeline:

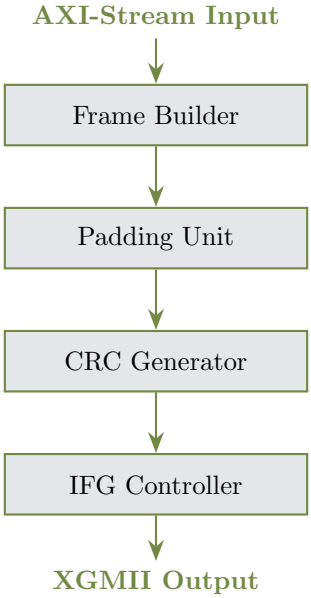

Figure 2: Transmit Datapath Pipeline

4.3.1 Ethernet Frame Format

The standard Ethernet frame format constructed by the TX path is as follows:

Preamble & SFD	MAC Header	Payload	FCS
8 Bytes	14 Bytes	Variable	4 Bytes

Table 5: Standard Ethernet Frame Format

Preamble and SFD: On the XGMII bus, the preamble consists of a Start control character (`0xFB`), followed by six bytes of standard preamble (`0x55`), and finishes with a single Start Frame Delimiter (SFD) byte (`0xD5`).

4.3.2 Transmit State Machine

The transmit module operates using a 7-state finite state machine to manage frame construction:

State	Description
`Idle`	Wait for packet arrival
`Payload`	Transmit packet payload
`Pad`	Insert padding if required
`Fcs1`	Transmit first CRC segment
`Fcs2`	Transmit remaining CRC
`Err`	Transmit error frame
`Ifg`	Enforce inter-frame gap

Table 6: Transmit Finite State Machine

4.3.3 Frame Padding

Ethernet frames must be at least 64 bytes long to ensure collision detection mechanisms function correctly on physical mediums. If the packet payload is smaller than this minimum frame size, the MAC automatically appends padding bytes before calculating and transmitting the CRC.

4.3.4 Parallel CRC Generation

Unlike serial implementations, the **Mac** core utilizes a bank of 8 parallel **LFSR** instances. During the elaboration phase, Chisel calculates the LFSR transition matrix to generate XOR-based hardware logic capable of processing up to 64 bits of data per cycle using the standard polynomial `0x04C11DB7`. This allows the core to calculate the Frame Check Sequence (FCS) for any valid *tkeep* combination in a single clock cycle.

4.3.5 Inter-Frame Gap (IFG)

The Ethernet standard requires a minimum delay between consecutive frames to allow network devices time to process the incoming packet. The transmit module enforces this by inserting idle control characters on the XGMII interface. The default inter-frame gap is **12 bytes**.

4.3.6 Deficit Idle Count (DIC)

To maintain the IEEE 802.3 mandated 12-byte average Inter-Frame Gap (IFG) while maximizing 10G throughput, the core implements a **Deficit Idle Count** register. This logic tracks "borrowed" idle bytes from the previous transmission, allowing the FSM to start the next packet after only 8 or 4 idle bytes if the historical average remains above 12.

4.3.7 Lane Alignment and Swapping

Ethernet 10G requirements dictate that the Start control character (`0xFB`) must be aligned to specific lanes. The TX path supports two alignment modes:

- **Lane 0 Alignment:** The packet starts on the first byte of the XGMII bus.
- **Lane 4 Alignment:** The packet starts on the fifth byte.

The *txStartPacket* status signal uses two bits to indicate the active alignment (`b01` for Lane 0, `b10` for Lane 4). When starting on Lane 4, the internal logic performs a 32-bit lane swap to maintain protocol compliance.

4.4 Receive Path Logic

The receive datapath processes Ethernet frames received from the XGMII interface, operating in the *rxClk* domain, and converts them into AXI-Stream packets.

Main functions:

- Frame detection

- Preamble validation
- Payload extraction
- CRC verification
- Packet classification

Receive Pipeline:

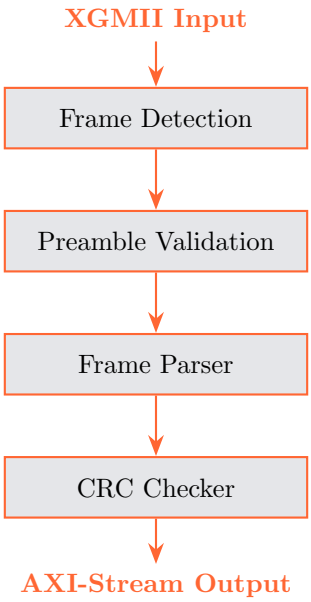

Figure 3: Receive Datapath Pipeline

4.4.1 Receive State Machine

The receive module orchestrates the decapsulation process using a concise 3-state finite state machine:

State	Description
`Idle`	Actively searching the XGMII bus for Start control characters.
`Payload`	Streaming active frame data to the AXI-Stream *mAxisRx* interface.
`Last`	Finalizing the packet termination specifically for Lane 4 aligned (swapped) frames.

Table 7: Receive Finite State Machine

4.4.2 Frame Detection and Preamble Validation

The receive module detects the start of frames using XGMII control characters:

Symbol	Value	Description
Idle	`0x07`	Idle character
Start	`0xFB`	Start of frame
Terminate	`0xFD`	End of frame
Error	`0xFE`	Error indication

Table 8: XGMII Control Characters

Once a Start character (`0xFB`) is detected on Lane 0 or Lane 4, the receiver verifies the Ethernet preamble sequence (`0x55` ... `0xD5`) before accepting frame data. Invalid preamble sequences trigger a preamble error flag.

4.4.3 Cross-Cycle Realignment

If a frame is detected on Lane 4, the RX path enters a "Swapped" state. It utilizes a 32-bit register (*swapRxdReg*) to buffer the lower half of the XGMII bus, realigning the payload across clock cycles so that the AXI-Stream interface always sees a Lane 0-aligned packet start.

4.4.4 CRC Residue Verification

The receive datapath computes the CRC of incoming frames using the same `0x04C11DB7` polynomial used during transmission. Because an Ethernet frame can terminate on any of the 8 XGMII lanes, the receiver validates the FCS by comparing the final LFSR state against a vector of 8 unique CRC residues. Each residue corresponds to a specific *XgmiiTerm* position within the 64-bit word. If the calculated CRC does not match the received FCS, the frame is marked as invalid.

4.4.5 Packet Classification

The MAC inspects destination MAC addresses to classify packets dynamically:

Packet Type	**Detection Method**
Unicast	LSB of destination address = 0
Multicast	LSB of destination address = 1
Broadcast	Destination address = `FF:FF:FF:FF:FF:FF`
VLAN	EtherType = `0x8100`

Table 9: Packet Classification Methods

4.5 Packet Statistics

The MAC generates runtime statistics for monitoring and debugging. These signals cross from the high-speed data domains into the slower *statClk* domain for software access.

Transmit Statistics:

Signal	**Description**
`statTxPktLen`	Length of transmitted packet
`statTxPktGood`	Valid transmitted packet
`statTxPktBad`	Packet transmission error

Table 10: Key Transmit Statistics

Receive Statistics:

Signal	Description
`statRxPktLen`	Length of received packet
`statRxPktGood`	Valid received packet
`statRxPktBad`	Packet error
`statRxErrBadFcs`	CRC error
`statRxErrOversize`	Oversized frame
`statRxErrFraming`	Framing error

Table 11: Key Receive Statistics

4.6 Precision Time Protocol (PTP)

When *ptpTsEn* is active, the core captures the value of the *ptpTs* input at the exact cycle the Start-of-Frame Delimiter (SFD) crosses the XGMII boundary.

- **ToD Format Support:** In Time-of-Day mode, the captured 96-bit timestamp is adjusted combinatorially to account for lane-swapping delays, ensuring nanosecond-level accuracy regardless of the starting lane.
- **Sideband Reporting:** The TX timestamp is returned via the *mAxisTxCpl* completion bus, while the RX timestamp is prepended to the *tuser* field of the *mAxisRx* interface.

4.7 Integration

To integrate the **Mac** core into a larger system:

- Connect application logic (or FIFOs) to the AXI-Stream TX and RX interfaces.
- Connect the MAC's XGMII interface directly to the PCS module.
- Configure the independent *txClk*, *rxClk*, and *statClk* domains alongside their respective reset signals.
- Enable optional features such as PTP timestamping if required by the application logic.

Figure 4 illustrates how the **Mac** core fits within a complete 10GBASE-R architecture, showing the flow of data from the application layer down to the physical medium and back up.

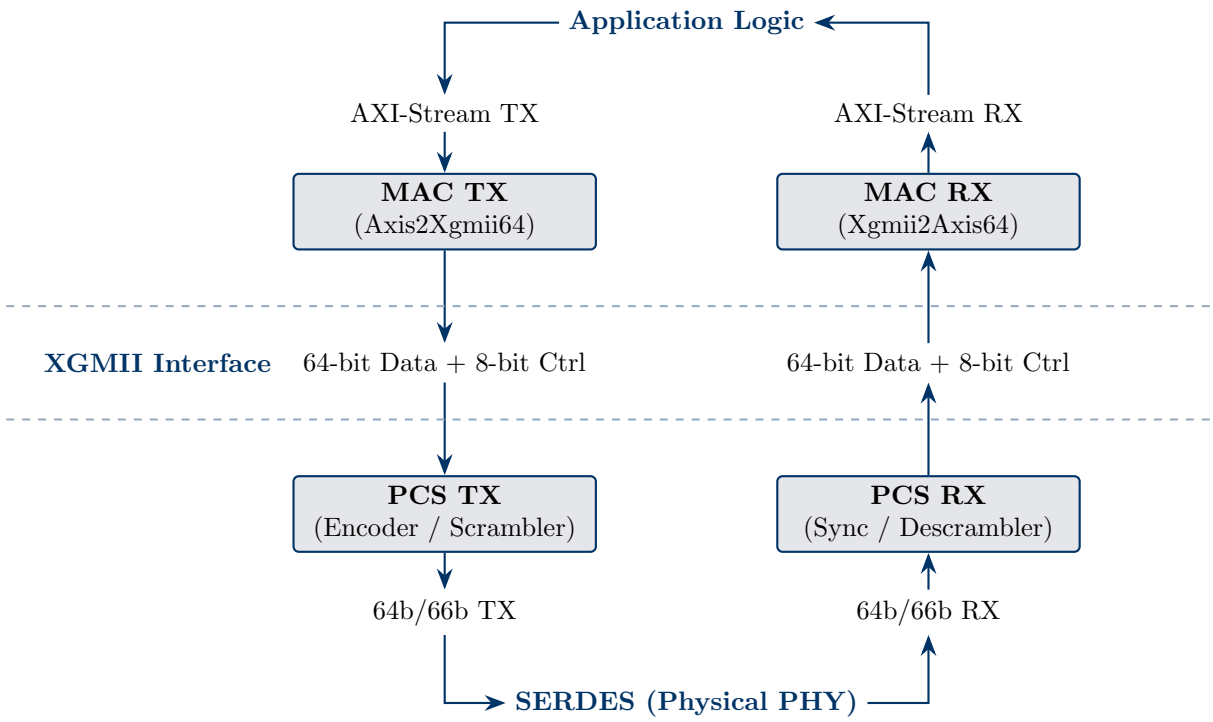

Figure 4: Combined MAC and PCS 10GBASE-R Architecture

5 Simulation

5.1 Tests

The included test bench tests the default configuration of the Mac. More configurations can be added as desired.

5.2 Code coverage

The tests exceed the chiselWare minimum standard of 90% Statement coverage and 95% Branch coverage.

5.3 Running simulation

Simulations can be run directly from the command prompt as follows:

```
$ sbt "project core" test
```

or from make as follows:

```
$ make test
```

6 Synthesis

6.1 Area

The **Mac** core has been tested in a number of configurations and the following results should be representative of what a user should see in their own technology.

Config Name	width	Gates
mac_default_no_stats[1]	64-bit	18,788

6.2 SDC File

An `.sdc` file is generated to provide synthesis and static timing analysis tools guidance for synthesis.

The `Mac.sdc` file can be found in the test case directory as follows:
`./modules/mac/generated/synTestCases/<test case name>/Mac.sdc`

6.3 Timing

The following timing was extracted using the generated .sdc files with the open-source Nangate 45nm library.

Config Name	Period	Duty Cycle	Input Delay	Output Delay	Slack
mac_default_no_stats[1]	6.40ns	50%	1.28ns	1.28ns	+3.04ns (MET)

Table 13: MAC Static Timing Analysis Constraints

[1] The default configuration is defined in the case class of `MacParams.scala`.

6.4 Multicycle Paths

None. The design is architected to complete all framing, CRC generation, and lane-alignment logic within a single clock cycle at 156.25 MHz.